Lab-3

Python Lists: Ordered Sequences

- **Core Logic:** A list is a linear, mutable (changeable) data structure. Elements are stored sequentially in memory and accessed via numerical indices (starting at 0). Lists are ideal when the order of data matters (like a timeline or procedural steps) and you need to iterate over items sequentially.
- **Memory & Performance:** Appending elements to the end is fast ($O(1)$), but inserting or deleting elements in the middle is computationally expensive ($O(n)$) because Python must shift all subsequent elements in memory to close the gap.

Python

```
# Syntax & Core Operations
ai_topics = ["Computer Vision", "NLP", "Robotics"]

# Access & Modify (Zero-indexed)
print(ai_topics[0])           # Output: Computer Vision
ai_topics[1] = "Deep Learning" # Modifies element in place

# Dynamic Sizing
ai_topics.append("Reinforcement Learning") # Adds to the end

# Slicing [start:stop:step]
subset = ai_topics[0:2]       # Grabs index 0 and 1
```

Python Dictionaries: Key-Value Mappings

- **Core Logic:** A dictionary represents a hash map. It maps an identifying *key* to a specific *value*, rather than placing items in a numbered sequence. Dictionaries are the go-to structure for establishing relationships between distinct properties (e.g., mapping a neural network layer name to its neuron count).
- **Memory & Performance:** Keys must be immutable (like strings or integers) and absolutely unique. Because Python computes a mathematical hash of the key to determine exactly where in memory the value lives, dictionary lookups, insertions, and deletions are virtually instant ($O(1)$), regardless of how massive the dictionary gets.

Python

```
# Syntax & Core Operations
model_config = {"name": "ResNet50", "accuracy": 0.92, "epochs": 100}

# Access & Modify via Keys
print(model_config["name"])    # Output: ResNet50
model_config["accuracy"] = 0.94 # Updates existing value

# Adding entirely new pairs
model_config["learning_rate"] = 0.001

# Safely extracting values without KeyError crashes
print(model_config.get("optimizer", "Not Found")) # Output: Not Found

# Iterating over key-value pairs efficiently
for key, value in model_config.items():
    print(f"{key} is set to {value}")
```

Feature	Python List []	Python Dictionary {}
Data Structure	Ordered sequence of individual items	Collection of linked key-value pairs
Access Method	Numerical index (e.g., `list[2]`)	Unique key (e.g., `dict["accuracy"]`)
Primary AI Use Case	Batches of data, chronological logs	Hyperparameter configs, graphs, vocabularies
Lookup Speed	$O(n)$ - scanning through elements	$O(1)$ - direct memory access via hash

Q 1: Model Evaluation Tracker (Lists within Dictionaries)

You are building an evaluation script for different machine learning models. You have a dictionary where the keys are model names and the values are lists of accuracy scores across 5 different validation folds. **Task:** Write a Python snippet to iterate through this dictionary, calculate the average accuracy for each model, and print the name of the model with the highest average.

```
# Starting Data
model_scores = {
    "LogisticRegression": [0.82, 0.85, 0.81, 0.86, 0.83],
    "RandomForest": [0.89, 0.91, 0.88, 0.92, 0.90],
    "NeuralNet": [0.95, 0.94, 0.93, 0.96, 0.95]
}
```

Q 2: Outlier Removal in Sensor Data (List Comprehension)

An autonomous vehicle's LiDAR sensor returns a list of distance measurements in meters. Occasionally, hardware glitches produce negative values or values exceeding 100 meters, which are physically impossible for this specific sensor. **Task:** Write a single line of Python code using list comprehension that takes a list called `raw_distances` and creates a new list

called `clean_distances` containing only the valid measurements (between 0 and 100, inclusive).

```
# Starting Data
raw_distances = [12.5, -3.2, 45.1, 105.8, 8.9, 0.0, 99.9, -0.1]
```

Q 3: Bag-of-Words Generator (Dictionary Counting)

For a basic Natural Language Processing (NLP) task, you need to count the frequency of words in a sentence to create a feature vector. **Task:** Write a function `build_vocab(text)` that takes a string, converts it to lowercase, splits it into words, and returns a dictionary where the keys are unique words and the values are their occurrence counts. You cannot use the `collections.Counter` module.

```
# Starting Data
text = "AI is transforming AI. The future of AI is now."
# Expected Output: {'ai': 3, 'is': 2, 'transforming': 1, 'the': 1,
'future': 1, 'of': 1, 'now.': 1}
```

Q 4: Heuristic Search Graph (Nested Structures) You are designing an A* search algorithm. The state-space graph is represented as a dictionary where each key is a node (state), and the value is a list of dictionaries representing connected neighbors and their step costs. **Task:** Write a Python snippet to add a new bidirectional connection between "Node_A" and "Node_D" with a cost of 5 to the existing graph. Ensure your code accounts for the possibility that "Node_D" might not exist in the graph yet.

```
# Starting Data
graph = {
    "Node_A": [{"node": "Node_B", "cost": 2}, {"node": "Node_C", "cost":
4}],
    "Node_B": [{"node": "Node_A", "cost": 2}],
    "Node_C": [{"node": "Node_A", "cost": 4}]
}
```

5: Merging Hyperparameter Configurations (Dictionary Updates) Your AI pipeline has a default configuration dictionary for a neural network. A user submits a custom configuration dictionary to override some of these defaults. **Task:** Write a snippet to update the `default_config` with the values from `user_config`. However, your code must *strictly* ignore any keys in `user_config` that do not already exist in `default_config` (to prevent injecting unsupported parameters).

```
# Starting Data
default_config = {"learning_rate": 0.01, "batch_size": 32, "epochs": 100,
"optimizer": "adam"}
user_config = {"learning_rate": 0.005, "epochs": 200, "dropout_rate": 0.2,
"use_gpu": True}
# Expected Output: default_config should update learning_rate and epochs,
but ignore dropout_rate and use_gpu.
```